Data Science Collective

Member-only story

Run Claude Code Locally on a Mac: 65 tok/s with a 4-bit Qwen3.6–27B and DFlash Speculative Decoding

Manjunath Janardhan Follow 11 min read · 1 day ago

A step-by-step guide to driving Claude Code entirely offline on Apple Silicon — no cloud, no API bill, ~19 GB of RAM, and real-time decode speeds thanks to MLX and block-diffusion speculative decoding.

Image By Manjunath Janardhan.

TL;DR — You can point Claude Code at a *local* Qwen 3.6 27-billion-parameter coding model running on your Mac. The trick is two pieces: `dflash serve` (an OpenAI-compatible MLX server that runs a 4-bit Qwen3.6–27B with DFlash speculative decoding) and `claude-code-router` (a bridge that translates Claude Code's Anthropic API into OpenAI calls, tool calls included). End result: ~65 tok/s average (up to ~120 on cached turns), ~19 GB resident, fully offline. This post walks you through it end to end.

Part 1 of a two-part series. This post is the offline setup; [Part 2] benchmarks it head-to-head against a sparse mixture-of-experts model — and the result flips a common assumption about what makes local coding fast.

See it in action— *3 min 33 sec clip at 6× speed (a real 21-minute session):* Claude Code driven entirely by the local 27B — three quick coding tasks, then a full FastAPI + React app built and run end to end. The long pause near the end is the ~4-minute context-explosion stall I

dissect later in this post.

"6× speed (21-min session)". three quick coding tasks, then a full FastAPI + React app built and run end to end

Why run Claude Code on a local model?

Claude Code is one of the best agentic coding tools available — but every request goes to the cloud, every token costs money, and nothing works on a plane. For experimentation, offline work, and privacy-sensitive code, it's genuinely useful to drive the same Claude Code CLI against a model running on your own machine.

The catch has always been speed. A 27B model decoding at 8 tok/s makes an agent loop unusable — Claude Code fires lots of short, tool-heavy turns, and you feel every one. The fix is **DFlash speculative decoding**, which gets a 4-bit Qwen3.6–27B to **~65 tok/s average and up to ~120 tok/s** on an Apple Silicon Mac. At that speed, a local 27B becomes a perfectly pleasant daily coding companion.

This guide shows you exactly how to wire it up.

What you'll build

Three moving parts:

1. **The model server** — `dflash serve` loads the 4-bit Qwen3.6–27B *target* plus a tiny 2B *draft* model and exposes an OpenAI-compatible endpoint.

2. **The bridge** — `claude-code-router` (CCR) sits in front and translates Claude Code's Anthropic-format requests into OpenAI-format calls, and translates the responses (text *and* tool calls) back.

3. **Claude Code itself** — launched via `ccr code`, talking to the bridge instead of the cloud.

How DFlash makes a 27B fast (the 30-second version)

DFlash(https://arxiv.org/abs/2602.06036) is **lossless** speculative decoding. A small *draft* model proposes a

block of γ tokens; the big *target* model verifies the whole block in a single forward pass; accepted tokens are kept, the first rejected one is corrected. Because verification is one batched forward instead of γ sequential ones, you get a big throughput win **with identical output quality** to running the target alone.

The 2B draft (`z-lab/Qwen3.6–27B-DFlash`) is *not* a standalone model — it's a purpose-built companion to the full Qwen3.6–27B, so acceptance rates are high (80–94% in practice, ~5.9 tokens accepted per cycle).

Why 4-bit MLX (and not a custom 3-bit quant)

I spent a while trying to beat this with a 3-bit codebook quant (TurboQuant) plus a hand-written verify kernel. It lost — and the reason is instructive.

Acceptance and tokens-per-cycle are basically identical — the draft proposes equally well against either target. The entire ~38× throughput gap is the **verify path**. MLX runs the hot loop with `mx.async_eval`, so the

memory-bound weight materialization of a native int4 matmul *overlaps* with the draft's compute and hides. A monolithic custom kernel sits squarely on the critical path and can't overlap, so it regresses end-to-end even though it wins in isolation.

The lesson: on Apple Silicon with MLX, **lean on the native int4 path and let async eval overlap the work.** Custom quantization earns its keep for *fitting* models that wouldn't otherwise load — not for squeezing a model that already fits.

Prerequisites

- **macOS on Apple Silicon**(M-series), with ≥ **~24 GB free RAM** (≈16 GB target + ≈3.5 GB draft + KV cache). Comfortable on 32–64 GB.

- **Python** with `dflash-mlx`(https://pypi.org/project/dflash-mlx) installed: `pip install dflash-mlx` (ideally in a venv).

- **Node ≥ 18**, then `npm i -g @musistudio/claude-code-router` (this guide uses CCR 2.0.0).

- **Claude Code CLI** (`claude`) on your `PATH`.

- The models cached locally (they'll auto-download on first run): `mlx-community/Qwen3.6–27B-4bit` and `z-lab/Qwen3.6–27B-DFlash`.

Step 1 — Start the OpenAI-compatible model server

```
dflash serve \
 -model mlx-community/Qwen3.6-27B-4bit \
 -draft-model z-lab/Qwen3.6-27B-DFlash \
 -host 127.0.0.1 -port 8787 \
 -verify-mode adaptive \
 -max-tokens 2048 \
 -chat-template-args '{"enable_thinking": false}'
```

Two flags matter:

- ` — **verify-mode adaptive**` shortens the verify block automatically when acceptance drops, which maximizes throughput across mixed prompts.

- ` — **chat-template-args ‘{“enable_thinking”: false}’**` turns OFF Qwen3.6’s `<think>` reasoning mode. **This is important for Claude Code.** With thinking on, the model emits a `reasoning_content` block and burns latency and tokens *before* it acts — bad for an agent loop. Off → direct content and crisp tool calls. (Want chain-of-thought back? Just drop the flag.)

Sanity check it:

```
curl -s localhost:8787/v1/chat/completions -d '{"model":"
# -> "dflash works"
# server log shows e.g. "decode 121.4 tok/s | 93.8% accep
```

The **first request is slow**— that’s kernel warm-up plus prompt prefill. Every request after that decodes fast.

Step 2 — Configure the bridge (claude-code-router)

Create `~/.claude-code-router/config.json`:

```
{
  "LOG": true,
  "HOST": "127.0.0.1",
  "Providers": [
    {
      "name": "dflash",
      "api_base_url": "http://127.0.0.1:8787/v1/chat/comp
      "api_key": "dflash-local",
      "models": ["mlx-community/Qwen3.6-27B-4bit"],
      "transformer": { "use": [ ["maxtoken", { "max_token
    }
  ],
  "Router": {
  "default": "dflash,mlx-community/Qwen3.6-27B-4bit",
  "background": "dflash,mlx-community/Qwen3.6-27B-4bit",
  "think": "dflash,mlx-community/Qwen3.6-27B-4bit",
  "longContext": "dflash,mlx-community/Qwen3.6-27B-4bit",
  "longContextThreshold": 30000,
  "webSearch": "dflash,mlx-community/Qwen3.6-27B-4bit"
  }
}
```

What the pieces do:

- **`enhancetool` transformer**— hardens tool-call formatting for a non-Claude model. This matters because Claude Code is **extremely** tool-heavy; `enhancetool` is what makes a tool request come back as a proper Anthropic `tool_use` block instead of malformed text.

- **`maxtoken`**— caps the response length the model/server will emit.

- **All routes → one model** — we only serve a single model, so `default`, `background`, `think`, `longContext`, and `webSearch` all point at it. Even Claude Code's lightweight `background` tasks hit the 27B, which is fine locally.

Then load the config:

```
ccr restart # picks up the config; bridge runs on :3456
ccr status
```

Step 3 — Launch Claude Code on the local model

```
ccr code # interactive Claude Code, driven by the local 2
OR
ccr code -p "…" # headless one-shot
```

Verify the whole chain end to end:

```
ccr code -p "Reply with exactly the three words: local mo
# -> local model online
```

That's it — Claude Code is now running entirely against a model on your Mac. Tool use works too: a `POST :3456/v1/messages` returns a `text` block for prose and a `tool_use` block for tool requests, exactly like the cloud API.

One-command bring-up

Once it works, wrap startup in a tiny idempotent script so you don't retype the commands. It checks whether `dflash serve` is already on `:8787`, starts it via `nohup` if not, restarts CCR, and prints the launch instructions:

```
#!/usr/bin/env bash
set -euo pipefail
SERVE_PORT=8787
TARGET="mlx-community/Qwen3.6-27B-4bit"
DRAFT="z-lab/Qwen3.6-27B-DFlash"
if ! curl -s -o /dev/null -m 2 "http://127.0.0.1:${SERVE_
  nohup dflash serve \
 -model "${TARGET}" -draft-model "${DRAFT}" \
 -host 127.0.0.1 -port "${SERVE_PORT}" \
 -verify-mode adaptive -max-tokens 2048 \
 -chat-template-args '{"enable_thinking": false}' \
> /tmp/dflash_serve.log 2>&1 &

for _ in $(seq 1 60); do
  curl -s -o /dev/null -m 2 "http://127.0.0.1:${SERVE_POR
sleep 2
done

fi

ccr restart || ccr start
echo "Ready -> run: ccr code"
```

Tear down: `ccr stop` and `pkill -f "dflash serve"`.

One thing worth knowing: CCR runs as its own daemon and survives your terminal, but a `dflash serve` you started *inside* a Claude Code session dies with that session. The `nohup` launcher above gives you a persistent server.

Performance: what to actually expect

- **Decode speed:** ~65 tok/s average across mixed prompts; up to ~120 tok/s on cached/coding turns (prefix-cache hits).

- **Acceptance:** ~80–94%, around 5.9 tokens accepted per verify cycle.

- **Memory:** ~19 GB resident total — ~16 GB target + ~3.5 GB draft + KV/prefix cache. The server prints its caps at startup (e.g. wired ~52 GB, prefix cache 8 GB).

- **Quality:** identical to running the 4-bit Qwen3.6–27B without DFlash — speculative decoding is lossless.

A genuinely capable coding model, fully local, at interactive speed, on a laptop.

Why it's actually fast — and the one thing that will stall it

Here's a number that should worry you: Claude Code's baseline prompt — system instructions, tool definitions, your `CLAUDE.md` — is already **~24,000 tokens** before you type anything. A 27B prefills at ~150–220 tok/s on this Mac, so prefilling 24k tokens from scratch takes **~2 minutes**. If every turn paid that, the agent loop would be unusable.

It isn't, because of **DFlash's prefix cache**. After the first turn, the server snapshots the KV state for that 24k prefix, and every subsequent turn only prefills the *new* tokens (your message + the latest tool results). In my own testing the cache "restored" prefill at **8,000–30,000**

tok/s, so 24–35k-token turns came back in **2–8 seconds**. The prefix cache is the unsung hero that makes a local agent viable at all.

But the cache has a cliff. When the prefix **diverges** (something early in the conversation changes) or gets **evicted** (the in-memory cache holds a limited number of entries), you fall back to a *full* prefill — and the cost is now `O(context)`.

I hit this hard while recording the demo for this very post. I asked the local 27B to *build and run* a full-stack FastAPI + React calculator app. It did — beautifully. Then I asked it to stop the two dev servers… and it went silent for four minutes. *(That's the long pause near the end of the demo video above.)*

It hadn't crashed. By that point the conversation had ballooned to **35,129 tokens** — every file it wrote, every tool result, plus the running servers' console output. That "stop the servers" turn was a cache miss, so the model re-prefilled all 35k tokens at ~143 tok/s ≈ **245 seconds** before it could emit a single token. (Memory was fine the whole time — ~21 GB peak. It was pure prefill.) The fix was instant: `/clear` reset the context to a few hundred tokens, and the next turn responded in 3 seconds.

Two operational lessons that will save you the same stall:

- `/clear` (or start a fresh session) before an unrelated request. Asking a 35k-token coding conversation to "stop the servers" makes it re-read 35k tokens first. A clean context answers instantly.

- **Don't build** *and* **run a big app in one session.** Long-lived dev-server output floods the context faster than anything else.

This raises an obvious question: if prefill is the real cost, is a *dense* 27B even the right model for local agentic coding? I benchmarked exactly that — see Part 2.

Sparse MoE vs Speculative Decoding: The Fastest Way to Run a Local Codin…

I benchmarked three ways to run a local agentic coding assistant on a Mac. The…

medium.com

Honest caveats

- **It's a 27B, not Claude.** Set expectations: weaker multi-step agentic reasoning, more tool-call mistakes, occasional format drift versus the real cloud Claude Code experience. `enhancetool` and thinking-off mitigate this, but they don't close the gap.

- **Thinking-off is a deliberate trade.** You're trading raw chain-of-thought reasoning for speed and tool reliability. Toggle it per-task via the chat-template arg.

- **One model serves every route.** If trivial `background` tasks feel sluggish, `ccr ui` lets you wire a separate small model just for those.

ccr ui. Image By Manjunath Janardhan

- **Switching models means restarting the server.** `dflash serve` loads one target at a time.

FAQ

Does this work on an Intel Mac or a non-Mac?

No. MLX and these int4 kernels are Apple Silicon only. On other hardware you'd use a different backend (e.g. llama.cpp or vLLM) behind the same claude-code-router bridge.

Will it fit in 16 GB?

It's tight. ~19 GB resident is the realistic floor with the draft and KV cache. 24 GB is workable, 32–64 GB is comfortable.

Is the output quality degraded by speculative decoding?

No. DFlash is lossless — the target model verifies every block, so you get exactly what the 4-bit model would produce on its own, just faster. The only quality reduction is from 4-bit quantization itself, which is mild.

Can I use this with Cursor or VS Code instead?

The same `dflash serve` + `claude-code-router` stack exposes a standard OpenAI endpoint, so any tool that accepts a custom base URL can point at it. This guide focuses on the Claude Code CLI.

Why claude-code-router and not just point Claude Code at the server?

Claude Code speaks the Anthropic Messages API; `dflash serve` speaks OpenAI Chat Completions. CCR translates between them — including the bidirectional tool-call format, which is the hard part.

Conclusion

With two open-source pieces — `dflash-mlx` for fast lossless speculative decoding and `claude-code-router` for the API bridge — you can run Claude Code against a 4-bit Qwen3.6–27B entirely on a Mac, at ~65 tok/s and ~19 GB of RAM. It won't replace Cloud Claude for hard agentic work, but it's a fast, free, offline coding model that's a good use case for IP-related code.

Next up — Part 2: DFlash makes this 27B *decode* fast. But is decoding even the bottleneck for agentic coding? I benchmarked this exact setup against a sparse mixture-of-experts (MoE) model and a pla in dense model, at context lengths from 2K to 24K. The result flipped my assumptions — and pointed at a simpler, bigger speedup than speculative decoding. → **[Sparse MoE vs Speculative Decoding: The Fastest Way to Run a Local Coding LLM on Apple Silicon]**

Sparse MoE vs Speculative Decoding: The Fastest Way to Run a Local Codin…

I benchmarked three ways to run a local agentic coding assistant on a Mac. The…

medium.com

Resources

- DFlash project page (Z Lab)
  https://z-lab.ai/projects/dflash/
- DFlash reference implementation on GitHub:
  https://github.com/z-lab/dflash
- mlx-community/Qwen3.6–27B-4bit on Hugging Face
  https://huggingface.co/mlx-community/Qwen3.6-27B-4bit
- Draft — z-lab/Qwen3.6–27B-DFlash on Hugging Face
  https://huggingface.co/z-lab/Qwen3.6-27B-DFlash

Support

If you found this article informative and valuable, I'd greatly appreciate your support:

"Give it a few claps 👏 on Medium to help others discover this content (did you know you can clap up to 50 times?). Your claps will help spread the knowledge to more readers."

- Share it with your network of AI enthusiasts and professionals.
- Subscribe to my YouTube channel for AI videos explained in simple English: https://www.youtube.com/@AIBroEnglish
- Connect with me on LinkedIn: https://www.linkedin.com/in/manjunath-janardhan-54a5537/

Thanks For Reading

Claude Code | Local Llm | Apple Silicon | Mlx | Speculative Decoding

Published in Data Science Collective

Follow

931K followers · Last published 1 day ago

Advice, insights, and ideas from the Medium data science community

Written by Manjunath Janardhan

Follow

1.1K followers · 87 following

AI/ML Computational Science Senior Manager at Accenture with 21+ years building enterprise AI, GenAI, and intelligent operations.